Tema 2

Instalación y uso de entornos de desarrollo

Entornos de desarrollo

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Funciones de un entorno de desarrollo 5
3. Tipos de entornos de desarrollo. Entornos de desarrollo libres t propietarios. Características 6
4. Instalación de un entorno de desarrollo 9
5. Uso básico de un entorno de desarrollo 10
5.1 Uso de herramientas y asistentes 10
5.2 Creación de proyectos 10
5.3 Incorporación de elementos a proyectos 11
5.4 Edición de programas. Sintaxis y formateo de código 12
5.5 Compilación de programas. Detección de errores 12
5.6 Generación de ejecutables 13
5.7 Ejecución de programas 13
5.8 Paneles y visitas 14
5.9 Personalización 15
5.10 Acceso a documentación 16
5.11 Configuración de actualizaciones 16
5.12 Automatización de tareas 16
6. Resumen 18
7. Glosario 19
8. Bibliografía 21

Tech Hub <FP> euroformac

Objetivos

- Conocer el concepto de entorno de desarrollo, así como sus funciones y componentes principales, aprendiendo a distinguirlos en función de sus características más relevantes.
- Obtener el conocimiento suficiente para la instalación y uso básico de un entorno de desarrollo concreto orientado a la programación en lenguaje Python: Pycharm.

Tech Hub <FP> euroformac

1. Introducción

Los entornos de desarrollo son la mejor herramienta para la creación de aplicaciones informáticas. En su interior, integran todo lo necesario para crear proyectos, editar códigos fuente, detectar errores o trabajar en equipo.

Hay multitud de entornos de desarrollo y es importante tener un buen conocimiento de estos para poder elegir el más adecuado para cada lenguaje y proyecto. Es conveniente disponer de información sobre sus características, tanto desde el punto de vista técnico como del relacionado con las licencias y los costes.

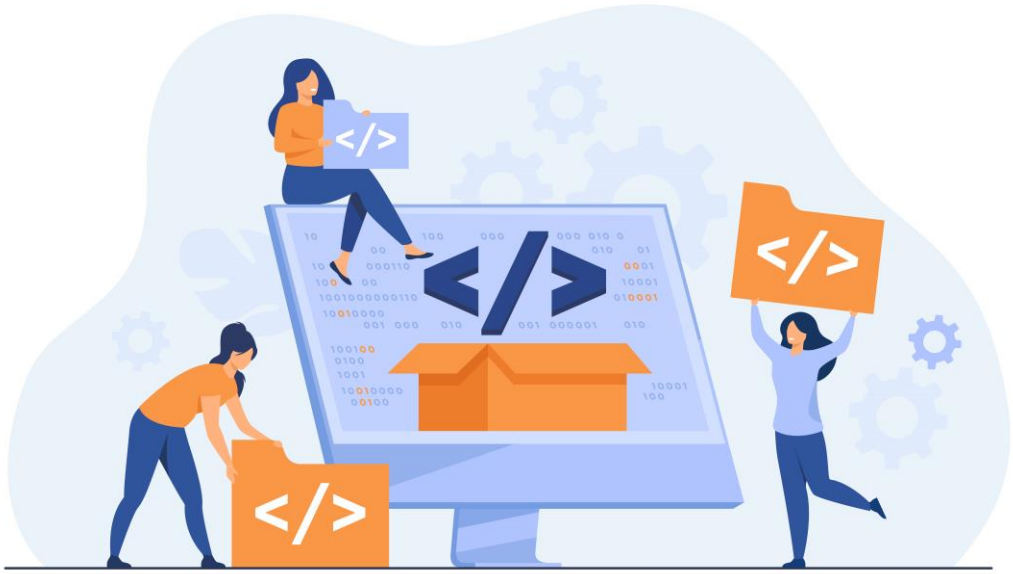
Tech Hub <FP> euroformac

2. Funciones de un entorno de desarrollo

En el pasado, las herramientas de programación eran simples editores que, en el mejor de los casos, ayudaban con la sintaxis del lenguaje de programación, indicando errores al igual que los correctores ortográficos en los procesadores de texto. Una vez escrito el código fuente con un editor, debía compilarse con otro programa (un compilador) y, en ocasiones, enlazarse con otro programa diferente (el enlazador o *linker*).

El desarrollo de aplicaciones, en la actualidad, implica muchas más tareas que la escritura y la compilación de programa: documentación, trabajo en equipo, control de versión, seguimiento de incidencias, manejo de librerías, generación de ejecutables o realización de pruebas son solo algunos ejemplos.

Los entornos de desarrollo integrados, popularmente conocidos por las siglas en inglés IDE (*Integrated Development Environment*), **son aplicaciones informáticas que integran las herramientas necesarias para desarrollar software, dando soporte a la mayoría de las actividades relacionadas con esta actividad.**



3. Tipos de entornos de desarrollo. Entornos de desarrollo libres t propietarios. Características

Existen muchos entornos de desarrollo. Algunos están dedicados a un lenguaje de programación concreto (como Android Studio para desarrollar aplicaciones Android). Otros cubren más lenguajes, como Eclipse o Visual Studio.

Sabías que

Visual Studio y Visual Studio Code no son el mismo entorno, aunque ambos son aplicaciones de Microsoft. Visual Studio es un IDE excepcionalmente completo y complejo, mientras que Visual Studio Code es un editor de código fuente.

A continuación, se enumeran algunos de los entornos de desarrollo más conocidos con los lenguajes a los que dan soporte.

IDE	Propietario	Lenguaje
Visual Studio	Microsoft	Multilenguaje
PyCharm	JetBrains	Python
IntelliJ IDEA	JetBrains	Java
Android Studio	Google	Android (Java, Kotlin, C y C++)
NetBeans	Apache Software Foundation	Multilenguaje (principalmente Java)
Eclipse	Eclipse Foundation	Multilenguaje (principalmente Java)
PhpStorm	JetBrains	PHP
Code:: Blocks	The Code:: Blocks team	C y C++

Por su parte, los editores de código modernos incorporan herramientas y, sin ser tan completos como los IDE, son suficientes en muchos casos para desarrollar aplicaciones. Habitualmente se utilizan para el desarrollo de aplicaciones web en el lado cliente (conocidas como *front*).

Sabías que

En el desarrollo de aplicaciones web se distingue entre la parte que se ejecuta en el servidor (*back*) y la parte que se ejecuta en el navegador o cliente (*front*). Se conoce como *fullstack* al conjunto formado por el *front* y el *back*.

La ventaja que tienen los editores de código frente a los entornos de desarrollo integrados es que son más ligeros y consumen menos recursos. Algunos de los más importantes se muestran en la siguiente tabla.

Editor	Propietario	Lenguaje
Visual Studio Code	Microsoft	Multilenguaje
Brackets	Adobe	JavaScript
Sublime Text 3	Sublime HP	Multilenguaje
Notepad++	Don Ho (Creador)	Multilenguaje
Ultraedit	Idera	Multilenguaje

Todos estos entornos de desarrollo y editores de código fuente pueden ser libres o propietarios, en función del tipo de licencia. La diferencia entre ambas categorías es que mientras el software libre proporciona acceso al código para estudiarlo, comprenderlo y mejorarlo, el software propietario restringe el uso que se puede dar del mismo

Más info

Richard Stallman es el fundador del movimiento del software libre, además de creador del sistema operativo GNU. Es una auténtica referencia de la programación y en el siguiente vídeo se puede conocer su visión del software propietario:

Ver enlace.

Además de la posibilidad de acceder a su código, los editores y los IDE pueden tener un coste de utilización o permitir su uso de manera gratuita. Cada fabricante o desarrollo de entornos tiene sus propias reglas en lo referente a si hay que pagar o no por utilizar sus productos.

Microsoft, por ejemplo, es una empresa conocida por sus productos de software comercial y propietario (no se permite acceder a su código) algunos tan célebres como *Windows* o el paquete ofimático Office. Sin embargo, proporciona acceso gratuito a editor *Visual Studio Code* así como al IDE Visual Studio para uso personal pero no para uso profesional. *JetBrains*, por su parte ofrece las versiones ***Community*** gratuita para uso personal y de código abierto y ***Professional*** para uso profesional con un coste anual por usuario, incluyendo diferentes funcionalidades una y otra versión.

Tech Hub <FP> euroformac

4. Instalación de un entorno de desarrollo

La instalación del entorno de desarrollo dependerá, por supuesto, del IDE elegido. Normalmente se debe descargar e instalar primero el compilador o el intérprete con el que se va a trabajar, aunque en ocasiones el propio instalador del IDE incluye la descarga e instalación de todas las herramientas necesarias para realizar el trabajo.

Tomando como ejemplo PyCharm, el IDE de desarrollo para proyectos Python, el proceso de instalación consistiría en seguir los siguientes pasos:

- Descarga la última versión del intérprete de Python y de sus herramientas a través del siguiente enlace: Ver enlace.
- Realizar la instalación de Pyhton. Se recomienda marcar la opción de añadir a la PATH la ruta de la instalación.
- Acceder a la sección de Pycharm de la web de la empresa JetBrains a través del siguiente enlace: Ver enlace.
- Pulsar el botón "Descargar" y seleccionar la opción de descarga de la versión *Community*.

Selección de la descarga de la versión Community de PyCharm

- Marcar las opciones de configuración deseadas y completar la instalación.
- En caso de no haber instalado previamente Pyhton, al crear el primer proyecto el entorno proporcionará la posibilidad de descargar e instalar las herramientas de este lenguaje, incluido su intérprete.

5. Uso básico de un entorno de desarrollo

Los entornos de desarrollo tienen una funcionalidad muy extensa por lo que resulta laborioso conocerla en profundidad. No obstante, en la mayoría de las ocasiones, no es necesario conocer más que un pequeño conjunto de acciones para desarrollar aplicaciones, especialmente en el ámbito académico.

5.1 Uso de herramientas y asistentes

Un entorno de desarrollo tiene múltiples herramientas. Por ejemplo, tiene herramientas para la gestión del control de versiones, la depuración de código o el entorno de ejecución.

Más info

Se puede obtener información referente a las herramientas proporcionadas por PyCharm en el siguiente enlace:

Ver enlace.

Además, los IDEs también proporcionan asistentes para realizar determinadas tareas. Un asistente habitual es de creación de proyectos, ayudando a los programadores a crear y configurar un proyecto a partir de sus características principales.

5.2 Creación de proyectos

En PyCharm, cuando se selecciona la creación de un nuevo proyecto, se muestra un panel desde el que realizar la confirmación de este.

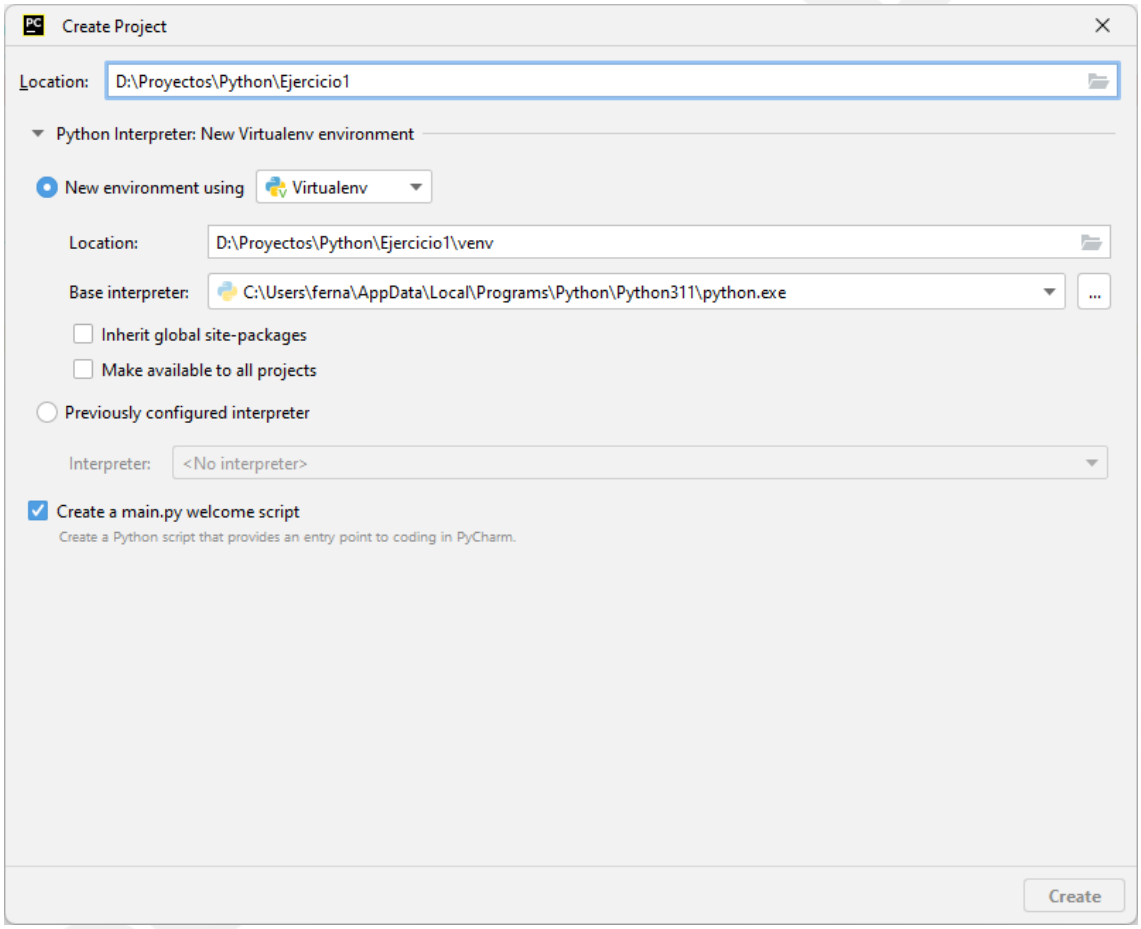
Lo primero que hay que indicar es la ubicación del proyecto. En la sección "*Location*" se debe seleccionar la carpeta en la que se quiere construir el proyecto. Si esta no existe, se creará.

El resto de las secciones se corresponden con la elección de la ubicación del entorno virtual, la versión de Python a utilizar y la creación de un fichero Python con el código básico de inicio de la ejecución.

Más info

Se puede aprender qué es un entorno virtual en Python y por qué es recomendable su uso en el siguiente enlace:

Ver enlace.



Pantalla de creación de proyectos con PyCharm

5.3 Incorporación de elementos a proyectos

Los elementos de un proyecto Python son ficheros escritos en este mismo lenguaje. Se conocen como **módulos** o **scripts** y tienen la extensión **.py**.

Para incorporar ficheros a un proyecto Python se dispone de dos alternativas principales: agregar un nuevo fichero (menú *File*, submenú *New…*, opción

Python file) o incluir un fichero ya existente (bastará con seleccionar el archivo desde el explorador de archivos, copiarlo y pegarlo en PyCharm seleccionando el nombre del proyecto).

5.4 Edición de programas. Sintaxis y formateo de código

Los IDEs (al igual que los editores de código fuente) proporcionan ayuda a la escritura de programas. Suelen proporcionar sugerencias y ayudas contextuales. Por ejemplo, sugiriendo el código que debería ir a continuación de un código ya escrito.

En la siguiente captura, el programador ha escrito el nombre de una lista seguido de un punto (dias.). El editor muestra una lista de los métodos u operaciones que se pueden hacer con las listas en Python.

```
dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"]
dias.
    m __init__(self)                                  list
    m sort(self, key, reverse)                        list
    m pop(self, __index)                              list
    m copy(self)                                      list
    m count(self, __value)                            list
    m append(self, __object)                          list
    m clear(self)                                     list
    m extend(self, __iterable)                        list
    m index(self, __value, __start, __stop)           list
    m insert(self, __index, __object)                 list
    m remove(self, __value)                           list
    m reverse(self)                                   list
    Press Intro to insert, Tabulador to replace  Next Tip
```

Sugerencia de código en PyCharm

5.5 Compilación de programas. Detección de errores

Los IDEs proporcionan información sobre los errores sintácticos, advirtiendo al programador tanto de su existencia como del motivo o motivos.

En la siguiente captura de pantalla se muestra una línea de código Python con un error (debería terminar con un corchete de cierre y termina con un paréntesis de cierre). Para advertir del error, PyCharm lo subraya en rojo y al situar el cursor encima muestra un mensaje informativo.

```
dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo")
```

']' or ',' expected

Detección de error sintáctico y su descripción

En los lenguajes de programación compilados, además de la ayuda en el editor, el proceso de compilación revisa y muestra los errores por pantalla, para permitir su corrección.

En el caso de los lenguajes interpretados, como Python, los errores se detectarán y mostrarán durante la ejecución. El siguiente texto es la salida de la ejecución de un programa con errores, donde se indica en qué línea se ha producido, en qué posición dentro de la línea y un mensaje descriptivo.

```
File "programa.py", line 3
fin_de_semana = ["Sábado","Domingo")
                                    ^
SyntaxError: closing parenthesis ')' does not match opening
parenthesis '['
```

5.6 Generación de ejecutables

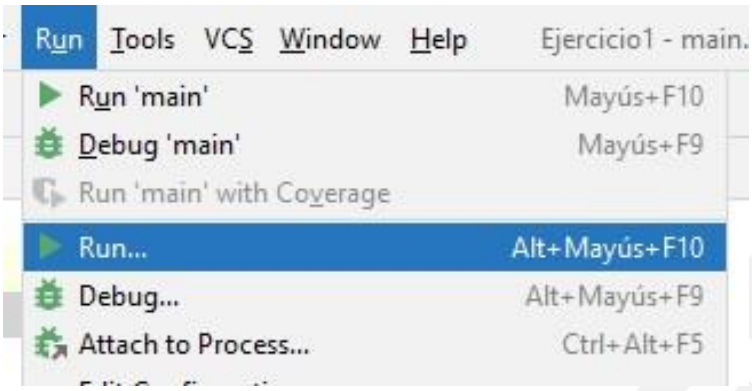
Los ejecutables son generados en los lenguajes compilados y, en ocasiones y mediante herramientas de terceros, en los lenguajes interpretados.

En Python no se suele generar ejecutables. Simplemente los ficheros de la aplicación se copian en la ubicación deseada y se ejecutan mediante la llamada al intérprete.

En otros lenguajes y entornos de desarrollo se deberá consultar la documentación técnica para saber cómo realizar el proceso de generación de ejecutables, ya que puede ser diferente en cada caso.

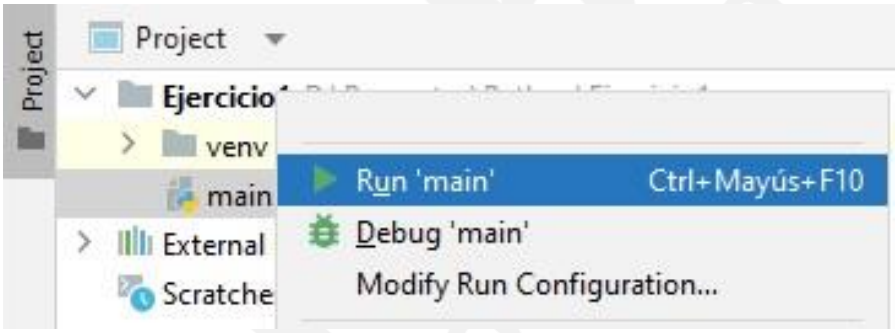
5.7 Ejecución de programas

Los entornos de desarrollo permiten la ejecución de los programas desde el propio editor. En PyCharm, se puede ejecutar el proyecto directamente pulsando sobre la opción **Run...** del menú **Run**.



Ejecución del proyecto desde PyCharm

También se puede ejecutar un fichero individualmente pulsando con el botón derecho sobre el fichero y seleccionando la opción **Run** seguida del nombre del fichero que aparece en el menú flotante.

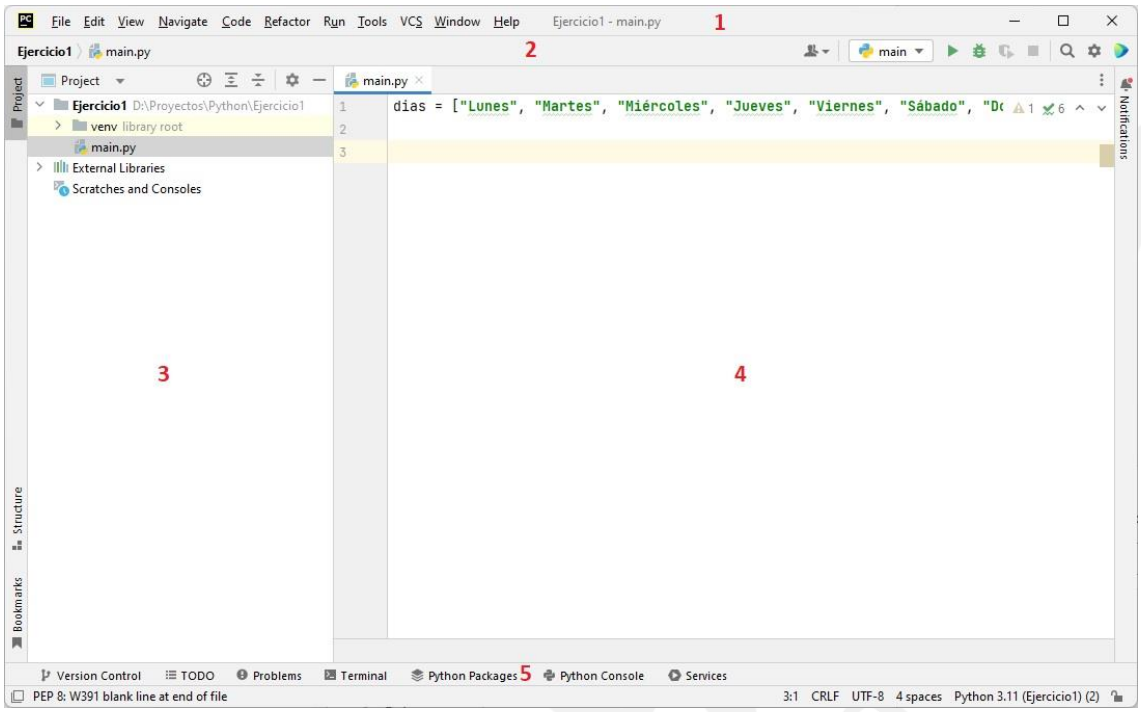


Ejecución de un fichero concreto

5.8 Paneles y visitas

Los entornos de desarrollo integrados (IDE) dividen la interfaz gráfica en paneles, pudiendo estos ser cambiados de ubicación, activados o desactivados según las necesidades del programador.

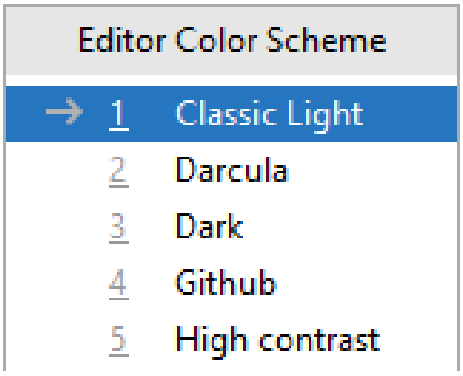
En PyCharm, por defecto, la interfaz gráfica se compone de los siguientes elementos principales (se pueden observar en la captura siguiente): 1 – Menú; 2 – Identificador de proyecto y de fichero y barra de acciones; 3 – Contenido del proyecto; 4 – Editor de código fuente; 5 – Acceso a diversos paneles, como la consola de Python, el terminal del sistema o la lista de acciones pendientes (*TODO list*).



Principales paneles de PyCharm

5.9 Personalización

Todos los IDE permiten personalizar el aspecto de sus elementos: tipo y tamaño de letra, disposición de los paneles o el esquema de color a utilizar. En PyCharm, el menú ***View*** da acceso a las opciones ***Appearance*** y ***Quick Switch Schema***. Desde esta última opción se puede modificar el esquema de color del entorno de desarrollo mediante la opción ***Editor Color Schema***.



Selector de esquema de color de PyCharm

5.10 Acceso a documentación

Los IDE son aplicaciones complejas y de mucho contenido, por lo que es conveniente aprender a utilizarlas en profundidad para sacarle el máximo provecho.

En PyCharm, la documentación se encuentra accesible desde el menú ***Help***. Las opciones más interesantes son ***Getting Started***, para aprender a utilizar el IDE y ***Help*** para buscar ayuda en problemas concretos.

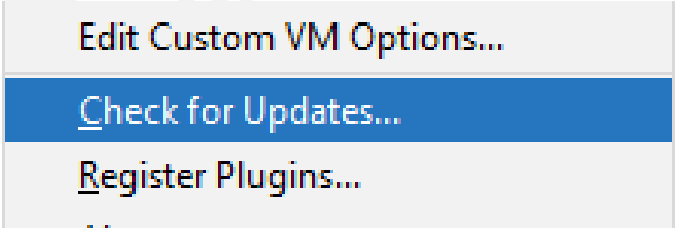
Instalación y desinstalación de aplicaciones, módulos y plugins adicionales

Los entornos de desarrollo suelen admitir mejoras en forma de elementos añadidos, llamados módulos o *plugins*. Depende totalmente del entorno el método para obtener dichos añadidos y para instalarlos y configurarlos. Pycharm no es una excepción, pudiéndose realizar el proceso de descarga e instalación desde la sección ***Plugins***, de la opción ***Settings*** dentro del menú ***File***.

5.11 Configuración de actualizaciones

La mayoría de los sistemas informáticos tiene errores o puntos de mejora que se van incorporando a lo largo del tiempo. Esto implica la necesidad de realizar actualizaciones para tener los programas en el mejor estado posible.

Cada entorno de desarrollo dispone de sus propios medios de actualización. En PyCharm, la actualización se realiza a través de la opción **Check for Updates**… del menú ***Help***.



Opción de actualización de PyCharm

5.12 Automatización de tareas

Las tareas que se repiten habitualmente tienden a automatizarse y los IDE suelen ofrecer ayudas para hacerlo.

Python, por ejemplo, proporciona herramientas de automatización para la creación, prueba y despliegue de los proyectos.

Más info

Se puede obtener más información relacionada con la automatización de las pruebas en PyCharm en el siguiente enlace:

Ver enlace.

6. Resumen

Un entorno de desarrollo es una aplicación informática que integra las herramientas necesarias para permitir la creación de un proyecto de software.

Los entornos de desarrollo se clasifican, en primer lugar, por el lenguaje o lenguajes de programación al que van destinados. Otras clasificaciones hacer referencia a su coste o la disponibilidad de acceder a su código fuente.

Cada entorno de desarrollo tiene sus propias particularidades, por lo que se debe aprender a utilizar cada uno de manera individual. No obstante, suelen compartir elementos, herramientas y técnicas comunes, por lo que la transición entre uno y otro no suele ser excesivamente problemática.

Tech Hub <FP> euroformac

Glosario

- **Ayuda contextual**. Es la ayuda que aparece en relación con un elemento concreto de contexto. En los editores de código fuente, es la ayuda que se muestra según se va escribiendo el código fuente.
- **Back**. En las aplicaciones web, es la parte de esta que se ejecuta en el servidor.
- **Entorno de Desarrollo Integrado (IDE)**. Sistema software que contiene un alto número de las herramientas necesarias para desarrollar software.
- **Entorno Virtual**. En Python, es un entorno de trabajo aislado que contiene una configuración específica para un proyecto.
- **Esquema de color**. Es una combinación determinada de colores de fondo y de texto.
- **Front**. En las aplicaciones web, es la parte de esta que se ejecuta en el cliente (navegador).
- **GNU**. Es el nombre de un sistema operativo y de un conjunto de programas que lo apoyan.
- **Interfaz Gráfica de Usuario (GUI)**. Son los elementos gráficos de los programas que permiten a los usuarios ver e interactuar con estos.
- **Licencia de software**. Es un contrato en el que se reflejan los derechos y las obligaciones del usuario de un sistema software.
- **Lista de tareas (*TO DO list*)**. Relación de trabajos, tareas o actividades que están pendientes de realizar. Se utiliza habitualmente en las labores de desarrollo de aplicaciones.
- **Panel**. Cada una de las secciones en las que está dividida la Interfaz Gráfica de Usuario (GUI) de un programa.
- **Plugin**. Es un entorno de desarrollo integrado, es un complemento que se instala junto con el entorno para ampliar o mejorar sus características.
- **Sistema de Control de Versiones**. Sistema para almacenar, gestionar y compartir los productos que se elaboran durante el desarrollo de software, ya sean estos ficheros de código fuente, documentos o cualquier tipo de recursos.
- **Software comercial o propietario**. Es el software que no permite acceder a su código fuente salvo por su propietario o creador.

- **Software libre**. Es el software que permite acceder a su código fuente para estudiarlo, modificarlo o mejorarlo.

Tech Hub <FP> euroformac

Bibliografía

- Install plugins. (n.d.). PyCharm Help. https://www.jetbrains.com/help/pycharm/managing-plugins.html
- Built-in Developer Tools (n.d.). PyCharm Help. https://www.jetbrains.com/pycharm/features/tools.html
- Auto-Run Tests. (n.d.). PyCharm Guide. https://www.jetbrains.com/pycharm/guide/tips/auto-run-tests/

Tech Hub <FP> euroformac